

Лабораторная работа № 2

Динамические модели SIR и Лотки–Вольтерры

Куокконен Дарина Андреевна

2026-08-29

Аннотация

В работе исследованы две нелинейные динамические системы. Для модели SIR определены базовое и эффективное репродуктивные числа, пик эпидемии и влияние интенсивности контактов. Для системы Лотки–Вольтерры найдены положения равновесия, построены временные и фазовые траектории, оценён период колебаний и проконтролировано сохранение первого интеграла. Расчёты выполнены в Julia, результаты представлены таблицами, графиками и выполненными блокнотами Jupyter; корректность реализации подтверждена автоматическими тестами.

Содержание

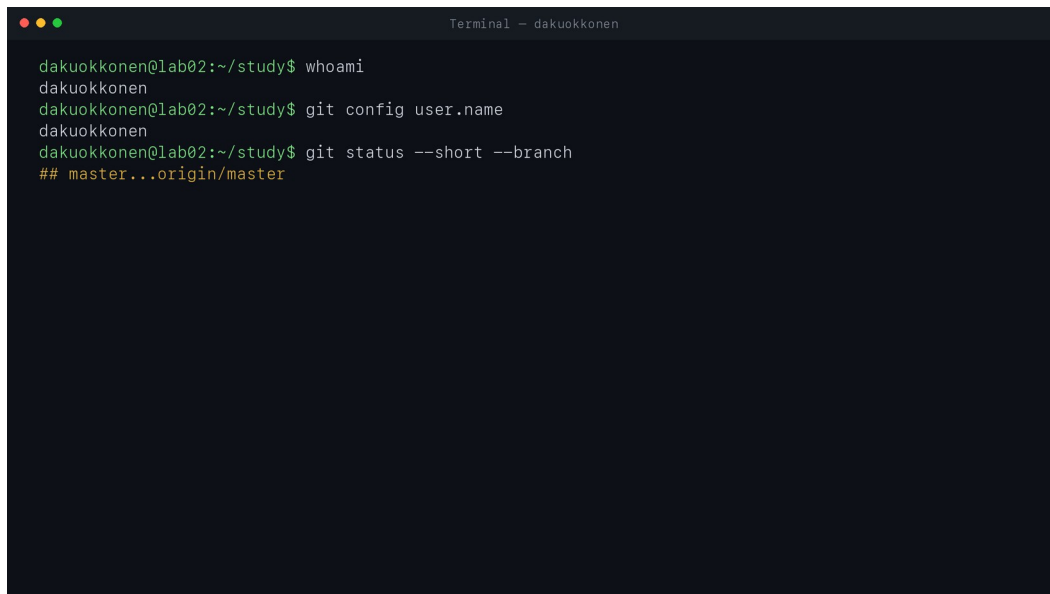
1. Цель и задание
2. Теоретическое введение
3. Организация вычислительного проекта
4. Исследование модели SIR
5. Исследование модели Лотки–Вольтерры
6. Производные форматы и Jupyter
7. Тестирование и контроль результатов
8. Воспроизводимость
9. Выводы
10. Источники

1. Цель и задание

Цель лабораторной работы — освоить численное исследование систем обыкновенных дифференциальных уравнений на примере эпидемиологической модели SIR и модели взаимодействия популяций Лотки–Вольтерры, а также подготовить воспроизводимый вычислительный проект средствами Julia.

В ходе работы требовалось:

1. создать отдельный Julia-проект и закрепить версии зависимостей;
2. реализовать правые части обеих систем ОДУ в общем модуле;
3. решить базовую задачу SIR, вычислить R_0 , момент и величину пика инфекции;
4. исследовать влияние среднего числа контактов на развитие эпидемии;
5. решить систему Лотки–Вольтерры, найти нетривиальное равновесие и оценить период колебаний;
6. построить семейство фазовых орбит и проверить сохранение первого интеграла;
7. сформировать CSV-данные, графики, QMD, выполненные IPYNB и HTML-документы;
8. проверить физические инварианты и численные свойства автоматическими тестами.

A screenshot of a terminal window titled "Terminal - dakuokkonen". The terminal shows the following commands and output:

```
dakuokkonen@lab02:~/study$ whoami
dakuokkonen
dakuokkonen@lab02:~/study$ git config user.name
dakuokkonen
dakuokkonen@lab02:~/study$ git status --short --branch
## master...origin/master
```

Рисунок 1: Проверка рабочего дерева и автора Git перед началом лабораторной работы

```
Terminal - dakuokkonen

dakuokkonen@lab02:~/study$ whoami
dakuokkonen
dakuokkonen@lab02:~/study$ git config user.name
dakuokkonen
dakuokkonen@lab02:~/study$ git status --short --branch
## master...origin/master
dakuokkonen@lab02:~/study$ git remote -v
gitverse git@gitverse-study:Dakuokkonen/study-2026-simulation-modeling.git (fetch)
gitverse git@gitverse-study:Dakuokkonen/study-2026-simulation-modeling.git (push)
origin git@github-study:dakuokkonen1/study-2026-simulation-modeling.git (fetch)
origin git@github-study:dakuokkonen1/study-2026-simulation-modeling.git (push)
```

Рисунок 2: Настроенные удалённые репозитории GitHub и GitVerse

2. Теоретическое введение

2.1 Эпидемиологическая модель SIR

Закрытая популяция постоянной численности N разделяется на три группы: $S(t)$ — восприимчивые к инфекции, $I(t)$ — инфицированные и способные передавать заболевание, $R(t)$ — выздоровевшие и выбывшие из цепочки передачи.

Использована система

$$\frac{dS}{dt} = -\beta c \frac{SI}{N}, (1)$$

$$\frac{dI}{dt} = \beta c \frac{SI}{N} - \gamma I, (2)$$

$$\frac{dR}{dt} = \gamma I. (3)$$

Здесь β — вероятность передачи инфекции при одном контакте, c — среднее число контактов за единицу времени, γ — интенсивность выздоровления. Суммирование правых частей (Уравнение 1)–(Уравнение 3) даёт

$$\frac{d}{dt}(S + I + R) = 0,$$

поэтому $S(t) + I(t) + R(t) = N$ является инвариантом модели.

Базовое репродуктивное число равно

$$R_0 = \frac{\beta c}{\gamma}. (4)$$

При $R_0 < 1$ малое возмущение затухает, а при $R_0 > 1$ инфекция сначала распространяется. С течением времени доля восприимчивых уменьшается, поэтому используется эффективное репродуктивное число

$$R_e(t) = R_0 \frac{S(t)}{N}. \quad (5)$$

Пик $I(t)$ достигается около момента, когда $R_e(t)$ пересекает единицу.

2.2 Модель Лотки–Вольтерры

Пусть $x(t)$ — численность жертв, а $y(t)$ — численность хищников. Классическая модель записывается как

$$\frac{dx}{dt} = \alpha x - \beta xy, \quad (6)$$

$$\frac{dy}{dt} = \delta xy - \gamma y. \quad (7)$$

Параметр α задаёт естественную скорость роста жертв, β — интенсивность их потерь при встрече с хищниками, δ — эффективность преобразования потреблённой пищи в прирост хищников, γ — естественную смертность хищников.

Нетривиальное равновесие определяется из условия равенства нулю обеих правых частей:

$$x^* = \frac{\gamma}{\delta}, y^* = \frac{\alpha}{\beta}. \quad (8)$$

Для положительных решений сохраняется первый интеграл

$$H(x, y) = \delta x - \gamma \ln x + \beta y - \alpha \ln y. \quad (9)$$

Его численный дрейф служит чувствительным контролем качества интегрирования. В отличие от SIR, система не стремится к конечному состоянию: в идеализированной модели возникают замкнутые фазовые орбиты вокруг точки (Уравнение 8).

3. Организация вычислительного проекта

Работа выполнена в той же изолированной среде, что и лабораторная №1: Julia 1.11.9, Quarto 1.10.18 и JupyterLab 4.4.7. Зависимости закреплены в Project.toml и Manifest.toml, а локальные каталоги пакетов исключены из Git.

```
Terminal - dakuokkonen
dakuokkonen@lab02:~/study$ cd labs/lab02/project
dakuokkonen@lab02:~/study/lab02$ julia --version
julia version 1.11.9
dakuokkonen@lab02:~/study/lab02$ quarto --version
1.10.18
dakuokkonen@lab02:~/study/lab02$ jupyter lab --version
4.4.7
```

Рисунок 3: Проверка версий Julia, Quarto и JupyterLab

Основные компоненты среды приведены в [Таблица 1](#).

Таблица 1: Основные зависимости проекта

Компонент	Назначение
DifferentialEquations.jl	постановка и решение систем ОДУ методом Tsit5
DrWatson.jl	организация путей, данных и активного проекта
DataFrames.jl, CSV.jl	табличные результаты и экспорт CSV
Plots.jl	построение временных и фазовых графиков
JLD2.jl	сохранение численных сводок вне публикуемого состава
Literate.jl	генерация Julia, QMD и IPYNB из литературного исходника
IJulia.jl	выполнение интерактивных блокнотов Julia
Quarto	формирование читаемой HTML-документации

В `src/epidemic_ecology.jl` собраны обе правые части, вычисление R_0 , $R_e(t)$, равновесия и первого интеграла. Благодаря этому сценарии и тесты используют одну реализацию математических формул.

```
module EpidemicEcology
```

```
export sir!, lotka_volterra!, basic_reproduction_number,
    effective_reproduction_number, lotka_equilibrium, lotka_invariant
```

```

function sir!(du, u, p, t)
    susceptible, infected, recovered = u
    beta, contacts, gamma = p
    population = susceptible + infected + recovered
    incidence = beta * contacts * susceptible * infected / population
    du[1] = -incidence
    du[2] = incidence - gamma * infected
    du[3] = gamma * infected
    return nothing
end

basic_reproduction_number(beta, contacts, gamma) =
    beta * contacts / gamma

effective_reproduction_number(susceptible, population, reproduction_number) =
    reproduction_number * susceptible / population

function lotka_volterra!(du, u, p, t)
    prey, predator = u
    alpha, beta, delta, gamma = p
    du[1] = alpha * prey - beta * prey * predator
    du[2] = delta * prey * predator - gamma * predator
    return nothing
end

lotka_equilibrium(alpha, beta, delta, gamma) =
    (gamma / delta, alpha / beta)

function lotka_invariant(pre, predator, alpha, beta, delta, gamma)
    return delta * prey - gamma * log(pre) +
        beta * predator - alpha * log(predator)
end

end
end

```

Листинг 1 — Полный модуль моделей src/epidemic_ecology.jl

```

1.10.18
dakuokkonen@lab02:~/study/lab02$ jupyter lab --version
4.4.7
dakuokkonen@lab02:~/study/lab02$ ls
Makefile Manifest.toml Project.toml data markdown notebooks plots scripts src test
dakuokkonen@lab02:~/study/lab02$ sed -n '1,80p' src/epidemic_ecology.jl
module EpidemicEcology

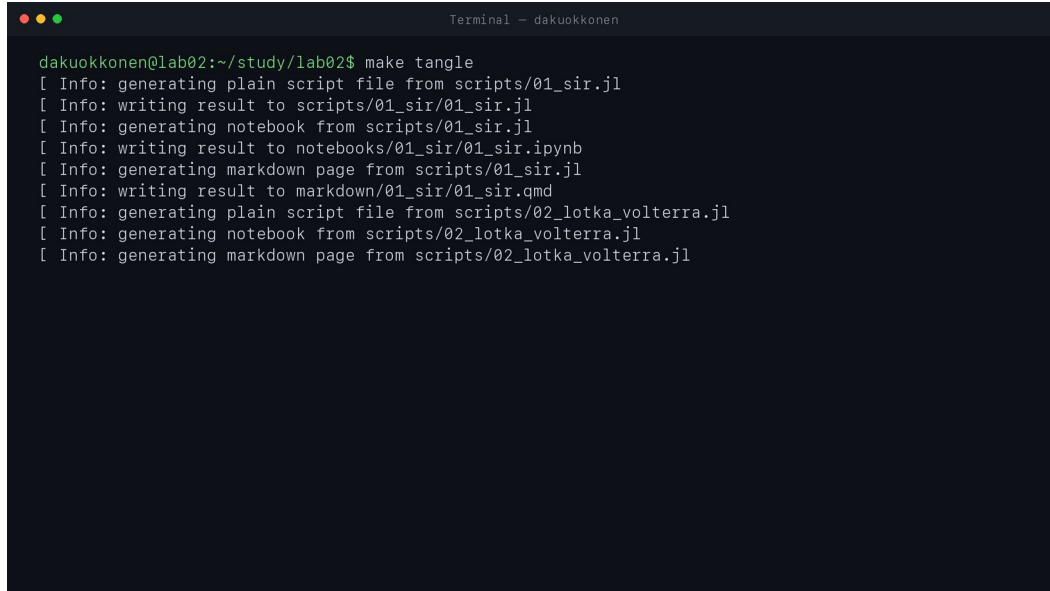
function sir!(du, u, p, t)
    susceptible, infected, recovered = u
    beta, contacts, gamma = p
    population = susceptible + infected + recovered
    incidence = beta * contacts * susceptible * infected / population
    du[1] = -incidence
    du[2] = incidence - gamma * infected
    du[3] = gamma * infected
end

function lotka_volterra!(du, u, p, t)
    prey, predator = u
    alpha, beta, delta, gamma = p
    du[1] = alpha * prey - beta * prey * predator
    du[2] = delta * prey * predator - gamma * predator
end
end

```

Рисунок 4: Фрагмент общего модуля двух моделей

Из литературных исходников автоматически создаются чистые скрипты, QMD и IPYNB. Процесс выполнен отдельно для 01_sir.jl и 02_lotka_volterra.jl.



```
Terminal - dakuokkonen
dakuokkonen@lab02:~/study/lab02$ make tangle
[ Info: generating plain script file from scripts/01_sir.jl
[ Info: writing result to scripts/01_sir/01_sir.jl
[ Info: generating notebook from scripts/01_sir.jl
[ Info: writing result to notebooks/01_sir/01_sir.ipynb
[ Info: generating markdown page from scripts/01_sir.jl
[ Info: writing result to markdown/01_sir/01_sir.qmd
[ Info: generating plain script file from scripts/02_lotka_volterra.jl
[ Info: generating notebook from scripts/02_lotka_volterra.jl
[ Info: generating markdown page from scripts/02_lotka_volterra.jl
```

Рисунок 5: Генерация производных форматов с помощью Literate.jl

4. Исследование модели SIR

4.1 Базовый сценарий

Начальные условия и параметры выбраны следующими:

$$(S_0, I_0, R_0^{state}) = (990, 10, 0),$$

$$\beta = 0.05, c = 10, \gamma = 0.25, t \in [0, 80].$$

Средняя продолжительность болезни равна $1/\gamma = 4$ дня, а по (Уравнение 4) получаем $R_0 = 2$. Сценарий создаёт ODEProblem, решает его методом Tsit5, формирует таблицу траектории и вычисляет диагностические показатели.

```
u0 = [990.0, 10.0, 0.0]
p = (beta=0.05, contacts=10.0, gamma=0.25)
tspan = (0.0, 80.0)

problem = ODEProblem(sir!, u0, tspan, (p.beta, p.contacts, p.gamma))
solution = solve(
    problem,
    Tsit5(),
    saveat=0.1,
    reltol=1e-9,
    abstol=1e-11,
)
```

```

trajectory = DataFrame(
    time=solution.t,
    susceptible=getindex.(solution.u, 1),
    infected=getindex.(solution.u, 2),
    recovered=getindex.(solution.u, 3),
)
trajectory.population = trajectory.susceptible .+
    trajectory.infected .+ trajectory.recovered

reproduction_number = basic_reproduction_number(
    p.beta, p.contacts, p.gamma
)
trajectory.effective_reproduction = effective_reproduction_number.(
    trajectory.susceptible,
    trajectory.population,
    reproduction_number,
)

peak_index = argmax(trajectory.infected)
peak_time = trajectory.time[peak_index]
peak_infected = trajectory.infected[peak_index]
cross_index = findfirst(<(1.0), trajectory.effective_reproduction)
threshold_time = isnothing(cross_index) ? NaN : trajectory.time[cross_index]
population_drift = maximum(
    abs.(trajectory.population .- first(trajectory.population))
)

CSV.write(datadir(script_name, "trajectory.csv"), trajectory)

```

Листинг 2 — Базовый расчёт и диагностика SIR в scripts/01_sir.jl

Результаты базового расчёта приведены в [Таблица 2](#).

Таблица 2: Численные характеристики базовой модели SIR

Показатель	Значение
Базовое репродуктивное число R_0	2,0000
Максимум $I(t)$	158,4512
Время пика	17,5 дня
Первое значение t , при котором $R_e < 1$	17,6 дня
Число выздоровевших к 80-му дню	800,1424
Максимальный дрейф $S + I + R$	$7,96 \cdot 10^{-13}$


```

Terminal - dakuokkonen

[ Info: generating plain script file from scripts/01_sir.jl
[ Info: writing result to scripts/01_sir/01_sir.jl
[ Info: generating notebook from scripts/01_sir.jl
[ Info: writing result to notebooks/01_sir/01_sir.ipynb
[ Info: generating markdown page from scripts/01_sir.jl
[ Info: writing result to markdown/01_sir/01_sir.qmd
[ Info: generating plain script file from scripts/02_lotka_volterra.jl
[ Info: generating notebook from scripts/02_lotka_volterra.jl
[ Info: generating markdown page from scripts/02_lotka_volterra.jl
dakuokkonen@lab02:~/study/lab02$ julia --project=. scripts/01_sir.jl
=== Базовый эксперимент SIR ===
R0 = 2.0
Пик I = 158.4512 при t = 17.5
Re < 1 с t = 17.6
R(80) = 800.1424
Максимальный дрейф S + I + R = 7.958078640513122e-13

=== Сканирование числа контактов ===
contacts R0 peak_infected peak_time final_recovered
3.0 0.6 10.0000 0.0 24.3649
5.0 1.0 10.0000 0.0 119.6710
7.5 1.5 69.7231 26.1 590.6630
10.0 2.0 158.4512 17.5 800.1424
15.0 3.0 303.7940 10.7 941.2020

```

Рисунок 6: Вывод базового и параметрического экспериментов SIR

На [Рисунок 7](#) видно монотонное уменьшение $S(t)$, колоколообразную кривую $I(t)$ и монотонный рост $R(t)$. Число инфицированных достигает максимума после того, как запас восприимчивых сократился достаточно, чтобы R_e приблизился к единице.

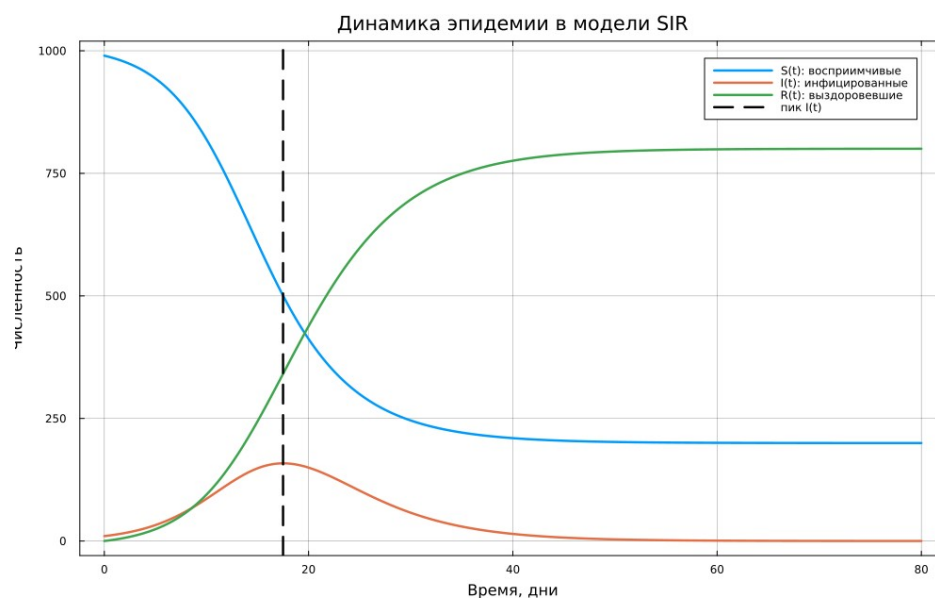


Рисунок 7: Динамика трёх групп в модели SIR

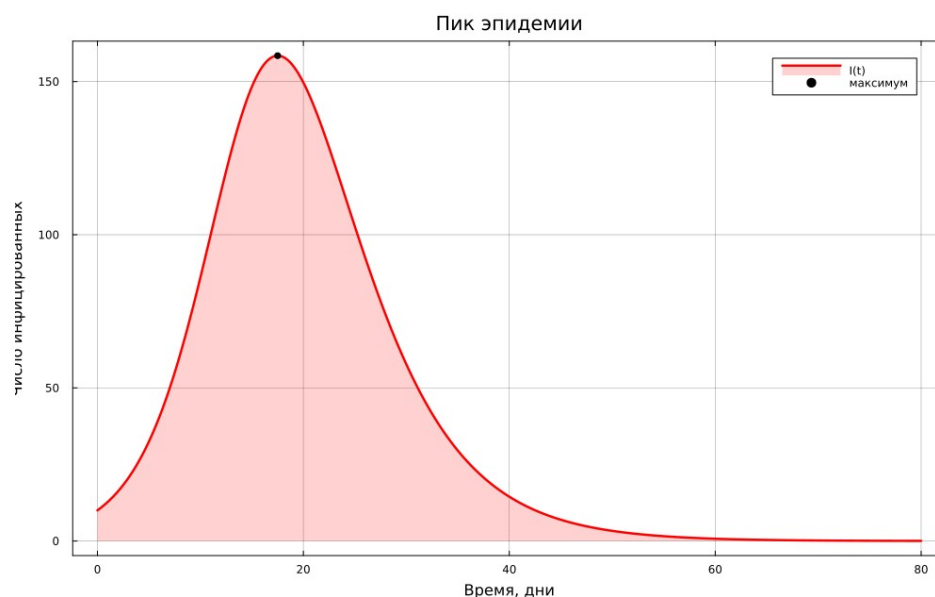


Рисунок 8: Пик числа инфицированных

График [Рисунок 9](#) подтверждает пороговую интерпретацию: после пересечения уровня $R_e = 1$ производная dI/dt становится отрицательной, и эпидемическая волна затухает.

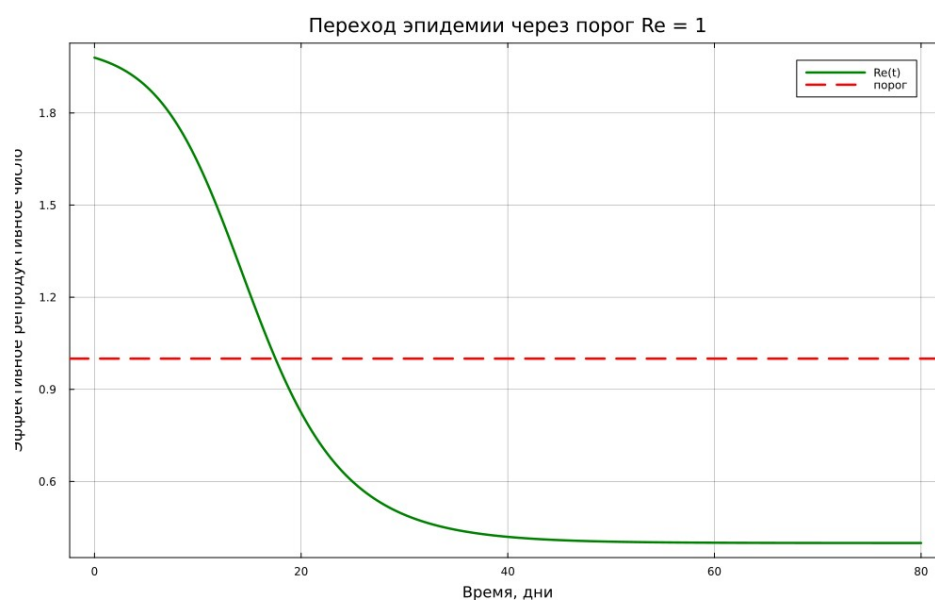


Рисунок 9: Динамика эффективного репродуктивного числа

Фазовая траектория в координатах (S, I) не замкнута. Восприимчивые только убывают, поэтому система перемещается от начального состояния к области малого числа инфицированных.

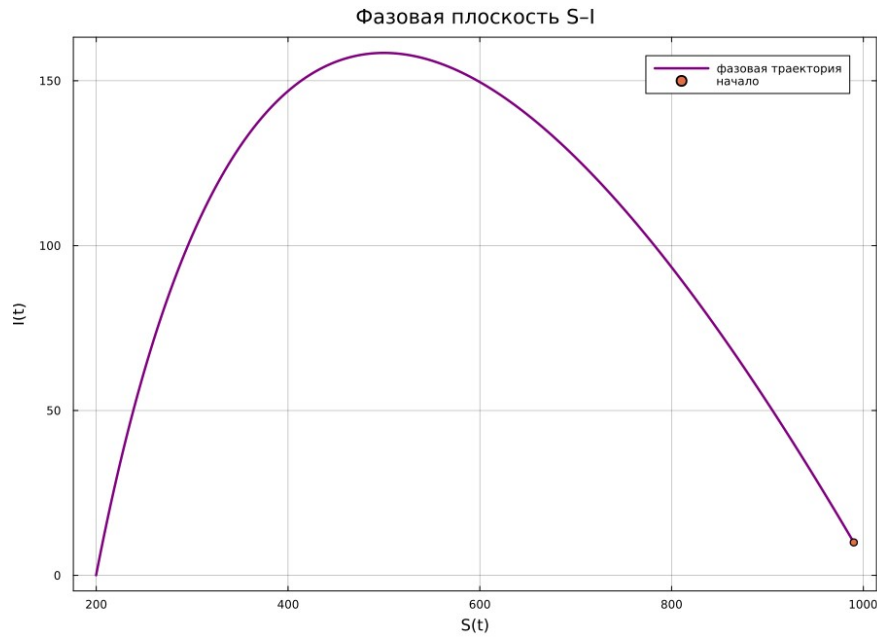


Рисунок 10: Фазовая траектория модели SIR

4.2 Исследование числа контактов

Параметр c изменялся по сетке $\{3; 5; 7,5; 10; 15\}$. Остальные параметры и начальные условия не менялись. Результаты представлены в [Таблица 3](#).

Таблица 3: Влияние числа контактов на ход эпидемии

c	R_0	Пик $I(t)$	Время пика	$R(80)$
3,0	0,6	10,0000	0,0	24,3649
5,0	1,0	10,0000	0,0	119,6710
7,5	1,5	69,7231	26,1	590,6630
10,0	2,0	158,4512	17,5	800,1424
15,0	3,0	303,7940	10,7	941,2020

При $R_0 = 0,6$ инфекция затухает сразу. Пороговый режим $R_0 = 1$ также не создаёт выраженного роста из-за того, что начальная доля восприимчивых меньше единицы. При увеличении числа контактов пик становится выше и возникает раньше.

Параметрический эксперимент выполняется циклом по пяти значениям c . Для каждого случая используется исходная задача `problem`, в которой заменяется только кортеж параметров; это исключает расхождение настроек решателя между вариантами.

```
contact_values = [3.0, 5.0, 7.5, 10.0, 15.0]
summary_rows = NamedTuple[]
```

```
for contacts in contact_values
    local_parameters = (p.beta, contacts, p.gamma)
    local_problem = remake(problem; p=local_parameters)
```

```

local_solution = solve(
    local_problem,
    Tsit5();
    saveat=saveat,
    reltol=1e-9,
    abstol=1e-11,
)
local_infected = getindex(local_solution.u, 2)
local_peak_index = argmax(local_infected)
local_r0 = basic_reproduction_number(
    p.beta, contacts, p.gamma
)
push!(summary_rows, (
    contacts=contacts,
    reproduction_number=local_r0,
    peak_infected=local_infected[local_peak_index],
    peak_time=local_solution.t[local_peak_index],
    final_recovered=getindex(last(local_solution.u), 3),
))
end

scan_summary = DataFrame(summary_rows)
CSV.write(datadir(script_name, "contact_scan.csv"), scan_summary)

```

Листинг 3 — Сканирование числа контактов в scripts/01_sir.jl

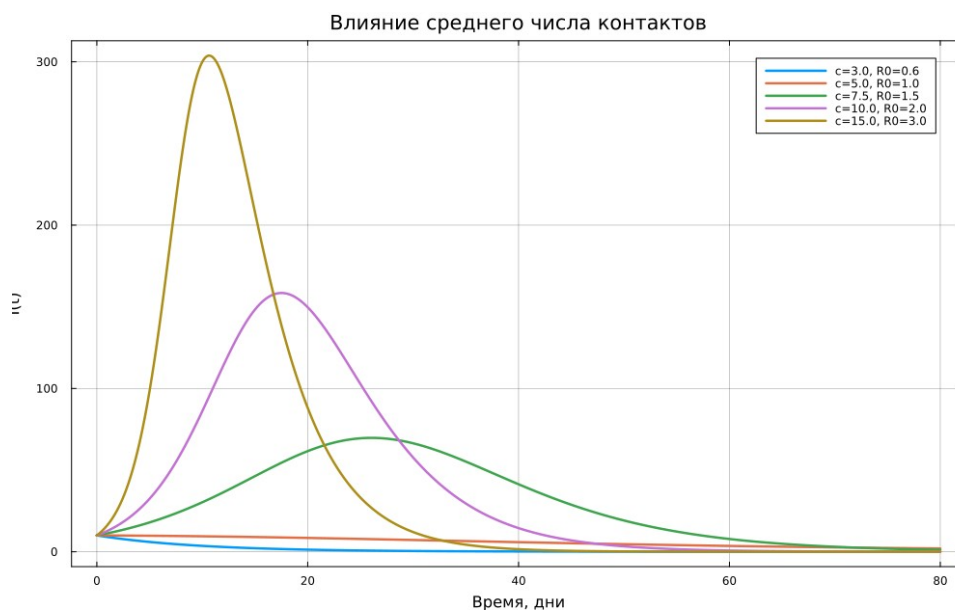


Рисунок 11: Сравнение кривых $I(t)$ при разных значениях числа контактов

5. Исследование модели Лотки–Вольтерры

5.1 Базовый сценарий

Для исследования выбраны параметры

$$\alpha=0.1, \beta=0.02, \delta=0.01, \gamma=0.3,$$

начальное состояние $(x_0, y_0) = (40, 9)$ и интервал $t \in [0, 400]$. По (Уравнение 8) стационарная точка равна

$$(x^*, y^*) = (30, 5).$$

```
p = (alpha=0.1, beta=0.02, delta=0.01, gamma=0.3)
u0 = [40.0, 9.0]
```

```
problem = ODEProblem(
    lotka_volterra!,
    u0,
    (0.0, 400.0),
    (p.alpha, p.beta, p.delta, p.gamma),
)
solution = solve(
    problem,
    Tsit5(),
    saveat=0.1,
    reltol=1e-10,
    abstol=1e-12,
)
```

Листинг 4 — Решение базовой системы в scripts/02_lotka_volterra.jl

Численные характеристики собраны в Таблица 4.

Таблица 4: Характеристики базовой модели Лотки–Вольтерры

Показатель	Значение
Равновесие жертв x^*	30,0000
Равновесие хищников y^*	5,0000
Диапазон $x(t)$	17,7539–46,8894
Диапазон $y(t)$	1,8952–10,4148
Оценка периода	37,6889
Дрейф первого интеграла	$1,82 \cdot 10^{-11}$

```

Terminal - dakuokkonen

dakuokkonen@lab02:~/study/lab02$ julia --project=. scripts/02_lotka_volterra.jl
=== Базовый эксперимент Лотки-Вольтерры ===
Равновесие (x*, y*) = (30.0, 5.0)
Диапазон x(t) = [17.7539, 46.8894]
Диапазон y(t) = [1.8952, 10.4148]
Оценка периода = 37.6889
Максимальный дрейф первого интеграла = 1.822053619093822e-11

prey0 predator0 prey_min prey_max predator_min predator_max
20.0      4.0    19.5524  43.6358     2.2873     9.3012
30.0     12.0    15.5703  51.3965     1.4569    12.0000
40.0      9.0    17.7539  46.8894     1.8952    10.4148
55.0      6.0    13.8884  55.3873     1.1506    13.4405

```

Рисунок 12: Результаты базового эксперимента и семейства начальных условий

На временном графике [Рисунок 13](#) рост жертв предшествует росту хищников. Увеличение числа хищников затем сокращает популяцию жертв, после чего из-за недостатка пищи уменьшается и популяция хищников. Ослабление давления позволяет жертвам восстановиться, и цикл повторяется.



Рисунок 13: Периодические колебания жертв и хищников

Фазовая траектория замкнута и охватывает стационарную точку (30,5). Начальное состояние не лежит в равновесии, поэтому оно определяет конкретную орбиту.

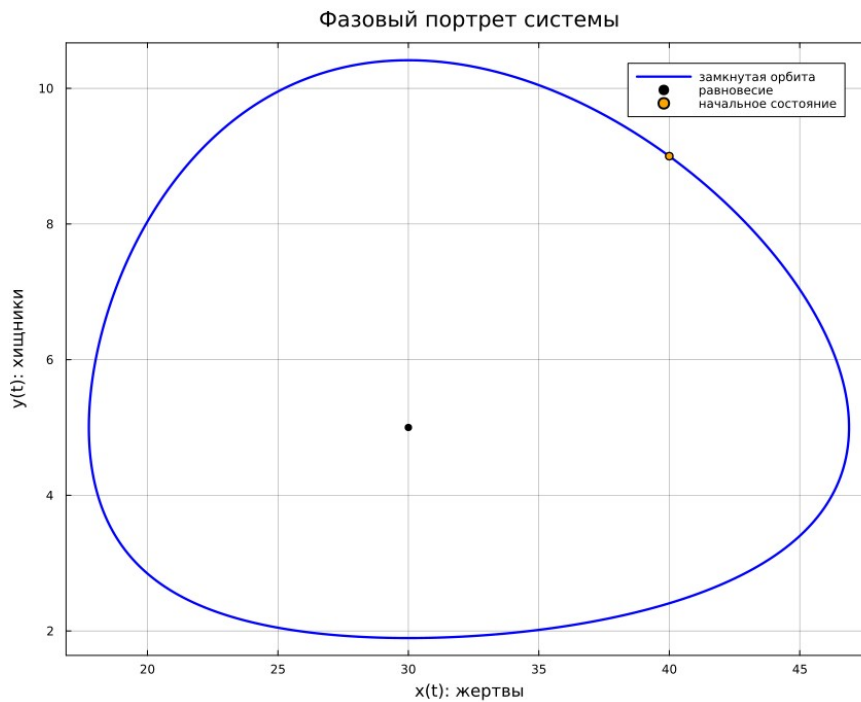


Рисунок 14: Фазовый портрет базовой модели

5.2 Контроль первого интеграла

Для каждой сохранённой точки вычислялось значение (Уравнение 9). Основная часть проверки приведена в листинге 4.

```
trajectory.invariant = lotka_invariant.(
    trajectory.prey,
    trajectory.predator,
    p.alpha,
    p.beta,
    p.delta,
    p.gamma,
)

equilibrium_prey, equilibrium_predator = lotka_equilibrium(
    p.alpha, p.beta, p.delta, p.gamma
)

prey_peak_indices = [
    index for index in 2:(nrow(trajectory) - 1)
    if trajectory.prey[index] > trajectory.prey[index - 1] &&
       trajectory.prey[index] >= trajectory.prey[index + 1]
]

period_estimate = length(prey_peak_indices) > 1 ?
    sum(diff(trajectory.time[prey_peak_indices])) /
    (length(prey_peak_indices) - 1) : NaN

invariant_drift = maximum(
    abs.(trajectory.invariant .- first(trajectory.invariant))
)
```

Листинг 5 — Равновесие, период и первый интеграл в scripts/02_lotka_volterra.jl

Максимальное отклонение порядка 10^{-11} существенно меньше масштаба самой функции $H(x, y)$. Следовательно, выбранные допуски решателя обеспечивают высокую численную точность на длинном интервале.

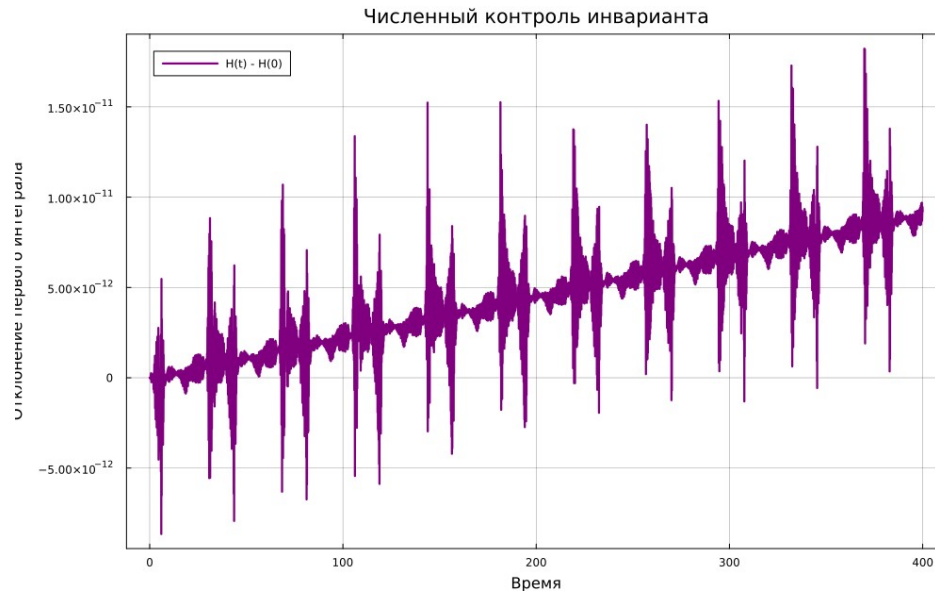


Рисунок 15: Отклонение первого интеграла от начального значения

5.3 Семейство начальных условий

Дополнительно исследованы четыре начальных состояния. Параметры модели были одинаковыми, поэтому все орбиты окружают одну и ту же точку равновесия, но имеют разные амплитуды.

```
initial_conditions = [  
    [20.0, 4.0],  
    [30.0, 12.0],  
    [40.0, 9.0],  
    [55.0, 6.0],  
]  
summary_rows = NamedTuple{  
  
for initial_state in initial_conditions  
    local_solution = solve(  
        remake(problem; u0=initial_state),  
        Tsit5(),  
        saveat=saveat,  
        reltol=1e-10,  
        abstol=1e-12,  
    )  
    local_prey = getindex.(local_solution.u, 1)  
    local_predator = getindex.(local_solution.u, 2)  
    local_invariant = lotka_invariant.(  
        local_prey,  
        local_predator,  
        p.alpha,
```



```

    p.beta,
    p.delta,
    p.gamma,
  )
  push!(summary_rows, (
    prey0=initial_state[1],
    predator0=initial_state[2],
    prey_min=minimum(local_prey),
    prey_max=maximum(local_prey),
    predator_min=minimum(local_predator),
    predator_max=maximum(local_predator),
    invariant_drift=maximum(
      abs.(local_invariant .- first(local_invariant))
    ),
  ))
end

initial_summary = DataFrame(summary_rows)
CSV.write(
  datadir(script_name, "initial_conditions.csv"),
  initial_summary,
)

```

Листинг 6 — Семейство начальных условий в scripts/02_lotka_volterra.jl

Таблица 5: Диапазоны колебаний при разных начальных условиях

x_0	y_0	x_{min}	x_{max}	y_{min}	y_{max}
20	4	19,5524	43,6358	2,2873	9,3012
30	12	15,5703	51,3965	1,4569	12,0000
40	9	17,7539	46,8894	1,8952	10,4148
55	6	13,8884	55,3873	1,1506	13,4405

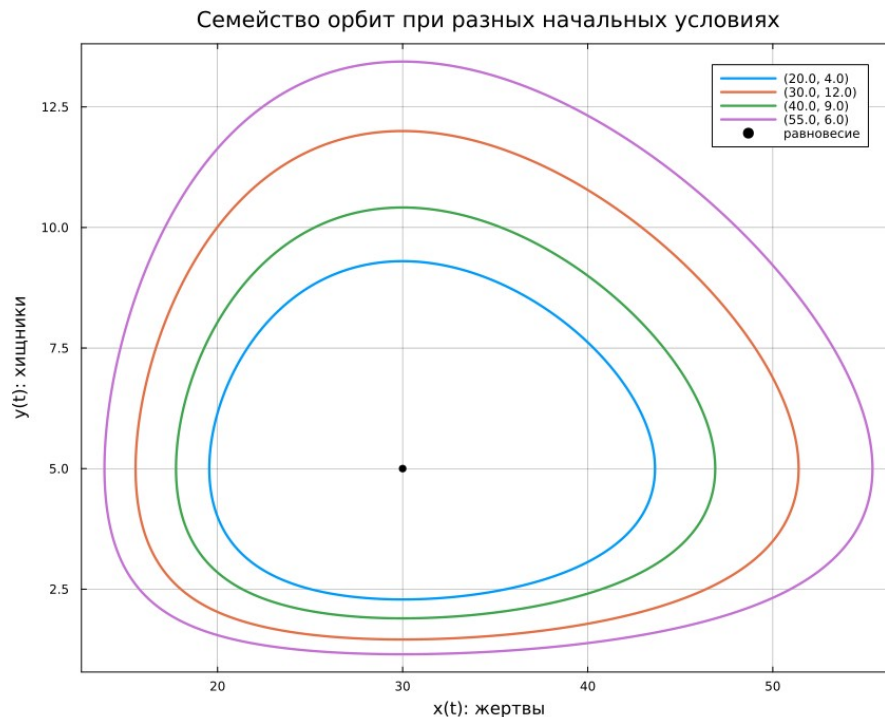


Рисунок 16: Семейство фазовых орбит

6. Производные форматы и Jupyter

После make tangle для каждого литературного исходника созданы три согласованных представления: исполняемый .jl, документ .qmd и блокнот .ipynb. Команда make notebooks выполнила оба блокнота с ядром Julia; номера выполнения и текстовые результаты сохранены внутри файлов.

```
using DrWatson
@quickactivate "project"
using Literate

function generate_formats(source::AbstractString)
    isfile(source) || error("Source file not found: $(source)")
    name = splitext(basename(source))[1]
    clean_dir = scriptsdir(name)
    quarto_dir = projectdir("markdown", name)
    notebook_dir = projectdir("notebooks", name)
    foreach(mkpath, (clean_dir, quarto_dir, notebook_dir))
    Literate.script(source, clean_dir; name=name, credit=false)
    Literate.notebook(
        source, notebook_dir; name=name, execute=false, credit=false
    )
    Literate.markdown(
        source,
        quarto_dir;
        name=name,
        flavor=Literate.QuartoFlavor(),
        credit=false,
    )
end
```

end

foreach(generate_formats, ARGS)

Листинг 7 — Генератор Julia, QMD и IPYNB scripts/tangle.jl

```
Terminal - dakuokkonen

Равновесие (x*, y*) = (30.0, 5.0)
Диапазон x(t) = [17.7539, 46.8894]
Диапазон y(t) = [1.8952, 10.4148]
Оценка периода = 37.6889
Максимальный дрейф первого интеграла = 1.822053619093822e-11

prey0 predator0 prey_min prey_max predator_min predator_max
20.0      4.0    19.5524  43.6358      2.2873      9.3012
30.0     12.0    15.5703  51.3965      1.4569     12.0000
40.0      9.0    17.7539  46.8894      1.8952     10.4148
55.0      6.0    13.8884  55.3873      1.1506     13.4405

dakuokkonen@lab02:~/study/lab02$ find data plots notebooks markdown -type f | sort
data/01_sir/contact_scan.csv
data/01_sir/trajectory.csv
data/02_lotka_volterra/initial_conditions.csv
data/02_lotka_volterra/trajectory.csv
markdown/01_sir/01_sir.html
markdown/01_sir/01_sir.qmd
markdown/02_lotka_volterra/02_lotka_volterra.html
markdown/02_lotka_volterra/02_lotka_volterra.qmd
notebooks/01_sir/01_sir.ipynb
notebooks/02_lotka_volterra/02_lotka_volterra.ipynb
plots/01_sir/sir_dynamics.png
plots/02_lotka_volterra/lotka_phase.png
```

Рисунок 17: Структура рассчитанных данных, графиков, QMD и IPYNB

```
Terminal - dakuokkonen

dakuokkonen@lab02:~/study/lab02$ make notebooks
[NbConvertApp] Converting notebook notebooks/01_sir/01_sir.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 11170 bytes to notebooks/01_sir/01_sir.ipynb
[NbConvertApp] Converting notebook notebooks/02_lotka_volterra/02_lotka_volterra.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 11257 bytes to notebooks/02_lotka_volterra/02_lotka_volterra.ipynb
```

Рисунок 18: Успешное выполнение обоих блокнотов через nbconvert

JupyterLab распознаёт специальное ядро лабораторной №2 и показывает все каталоги проекта.

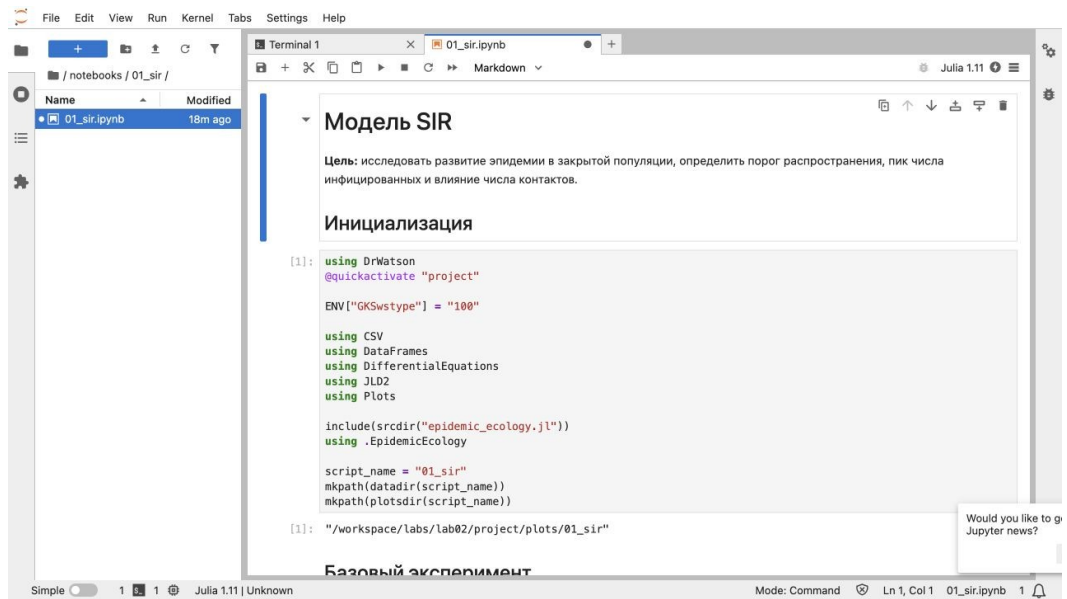


Рисунок 19: Интерфейс JupyterLab и ядро Julia лабораторной №2

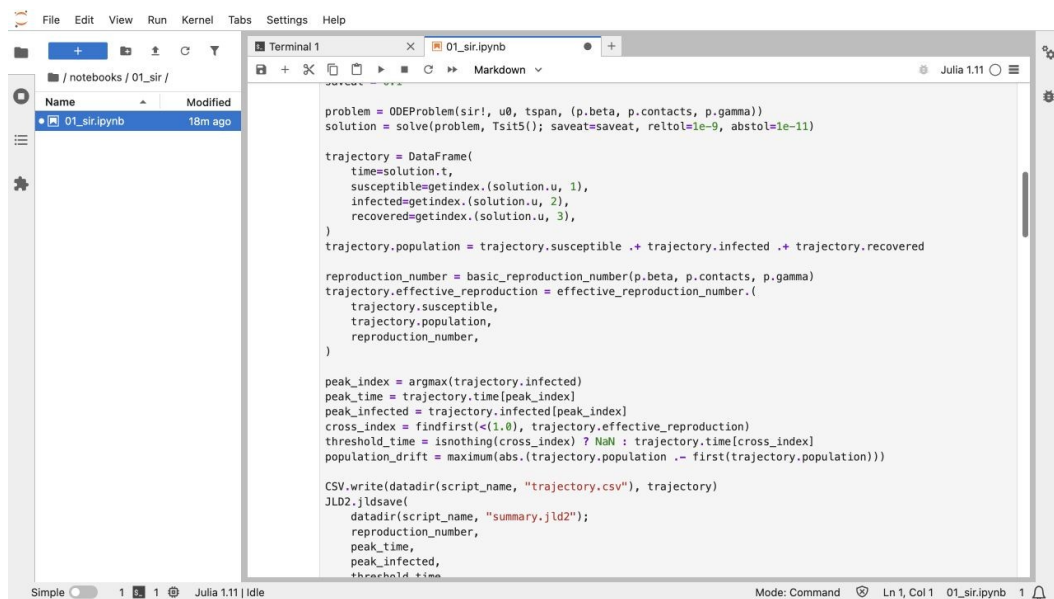


Рисунок 20: Начало выполненного блокнота модели SIR

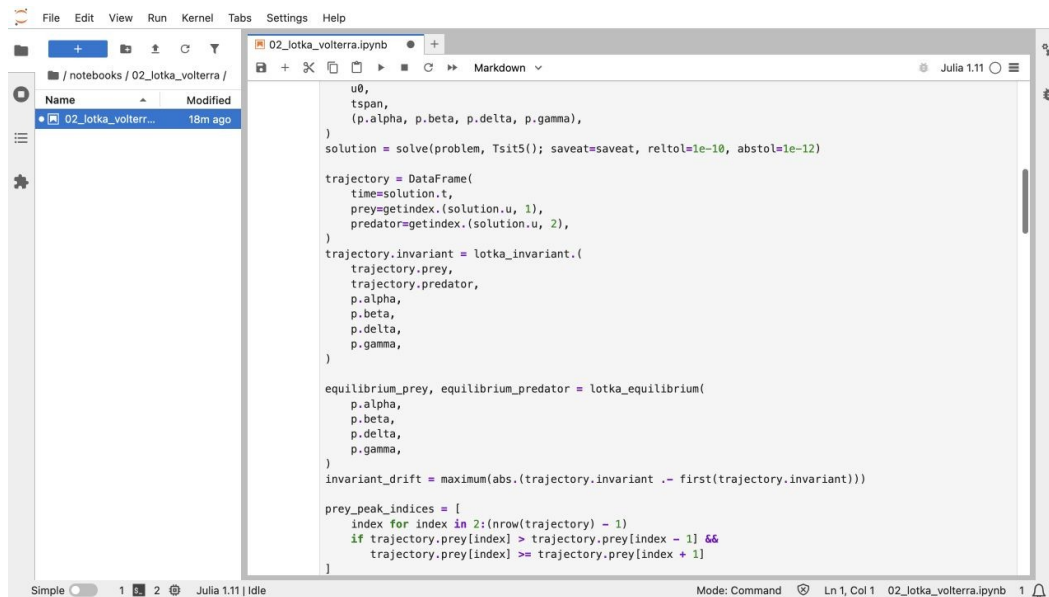


Рисунок 21: Начало выполненного блокнота модели Лотки–Вольтерры

7. Тестирование и контроль результатов

Автоматические тесты проверяют не только отдельные функции, но и свойства численных траекторий:

- сумма правых частей SIR равна нулю;
- $S + I + R$ сохраняется с точностью лучше 10^{-7} ;
- все компоненты SIR неотрицательны;
- при $R_0 = 2$ число инфицированных действительно растёт выше начального;
- аналитическое равновесие Лотки–Вольтерры равно $(30, 5)$;
- в равновесии обе производные равны нулю;
- траектория остаётся в положительном квадранте;
- первый интеграл сохраняется с точностью лучше 10^{-7} .

@testset "SIR model" begin

```

du = zeros(3)
state = [990.0, 10.0, 0.0]
sir!(du, state, (0.05, 10.0, 0.25), 0.0)
@test sum(du) ≈ 0.0 atol=1e-12
@test basic_reproduction_number(0.05, 10.0, 0.25) ≈ 2.0
@test effective_reproduction_number(500.0, 1000.0, 2.0) ≈ 1.0

problem = ODEProblem(
    sir!, state, (0.0, 80.0), (0.05, 10.0, 0.25)
)
solution = solve(
    problem, Tsit5(); saveat=0.1, reltol=1e-9, abstol=1e-11
)
totals = sum.(solution.u)
@test maximum(abs.(totals .- first(totals))) < 1e-7
@test minimum(minimum.(solution.u)) >= -1e-8

```

```

    @test maximum(getindex.(solution.u, 2)) > state[2]
end

@testset "Lotka–Volterra model" begin
    equilibrium = lotka_equilibrium(0.1, 0.02, 0.01, 0.3)
    @test collect(equilibrium) ≈ [30.0, 5.0]

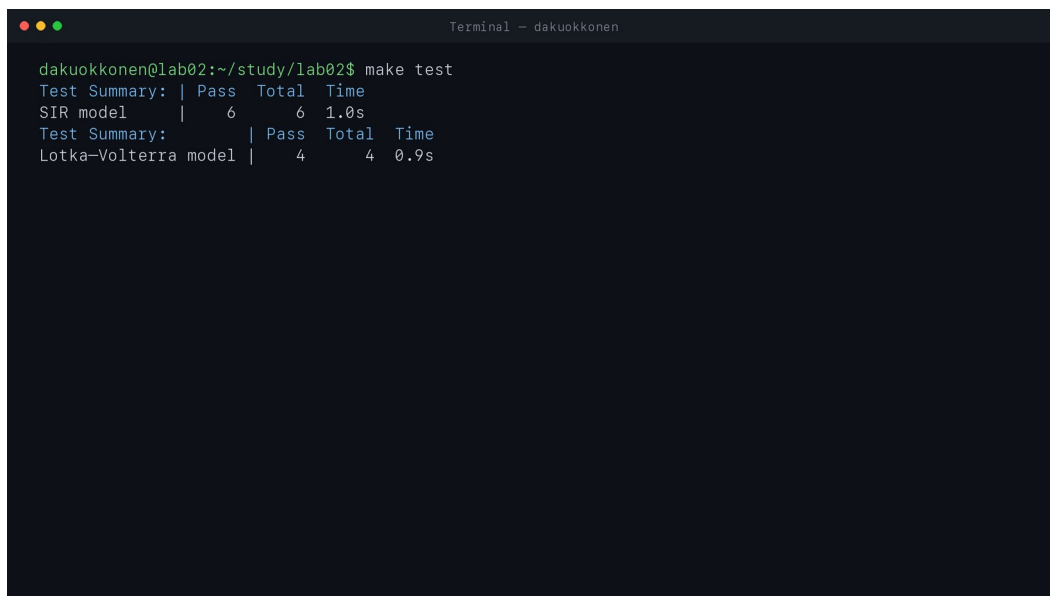
    du = zeros(2)
    lotka_volterra!(
        du, collect(equilibrium), (0.1, 0.02, 0.01, 0.3), 0.0
    )
    @test du ≈ zeros(2) atol=1e-12

    problem = ODEProblem(
        lotka_volterra!,
        [40.0, 9.0],
        (0.0, 100.0),
        (0.1, 0.02, 0.01, 0.3),
    )
    solution = solve(
        problem, Tsit5(); saveat=0.1, reltol=1e-10, abstol=1e-12
    )
    values = lotka_invariant.(
        getindex.(solution.u, 1),
        getindex.(solution.u, 2),
        0.1, 0.02, 0.01, 0.3,
    )
    @test maximum(abs.(values .- first(values))) < 1e-7
    @test minimum(minimum.(solution.u)) > 0.0
end

```

Листинг 8 — Полный состав проверок в test/runtests.jl

Всего выполнено 10 проверок: шесть для SIR и четыре для системы Лотки–Вольтерры. Все тесты завершились успешно.



```

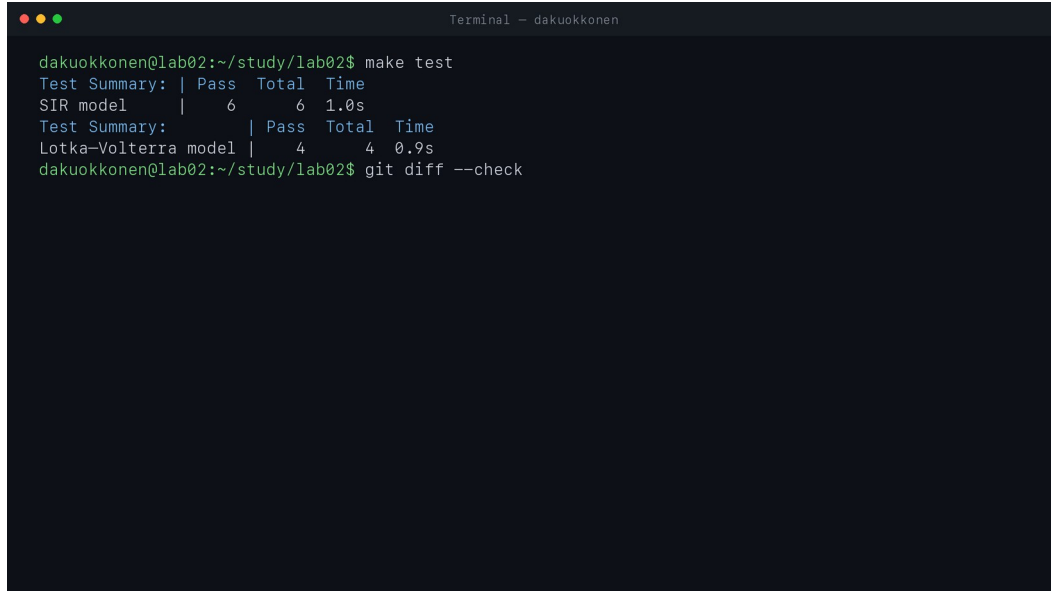
Terminal - dakuokkonen

dakuokkonen@lab02:~/study/lab02$ make test
Test Summary: | Pass  Total  Time
SIR model    |    6     6   1.0s
Test Summary: | Pass  Total  Time
Lotka-Volterra model |    4     4   0.9s

```

Рисунок 22: Полностью пройденный набор тестов

Перед фиксацией результатов дополнительно выполняется `git diff --check`, проверяется состав лабораторной и отсутствие локальных служебных файлов в Git.

A terminal window titled "Terminal - dakuokkonen" showing the output of a 'make test' command. The output displays two test summaries. The first summary is for the 'SIR model', showing 6 tests passed out of 6, taking 1.0s. The second summary is for the 'Lotka-Volterra model', showing 4 tests passed out of 4, taking 0.9s. Below the summaries, the command 'git diff --check' is entered at the prompt.

```
dakuokkonen@lab02:~/study/lab02$ make test
Test Summary: | Pass  Total  Time
SIR model     |    6      6  1.0s
Test Summary: | Pass  Total  Time
Lotka-Volterra model |    4      4  0.9s
dakuokkonen@lab02:~/study/lab02$ git diff --check
```

Рисунок 23: Проверка форматирования изменений

8. Воспроизводимость

Рабочая запись начинается уже внутри контейнера в каталоге `/workspace/labs/lab02/project`. Полный расчёт повторяется последовательно:

```
make tangle
make run
make notebooks
make docs
make test
```

Те же этапы объединены целью `make all`. Полный Makefile, который используется в лабораторной работе, приведён в листинге 9.

```
.PHONY: setup tangle run notebooks docs test all clean-results
```

```
setup:
```

```
    julia --project=. scripts/setup_environment.jl
```

```
tangle:
```

```
    julia --project=. scripts/tangle.jl scripts/01_sir.jl
```

```
    julia --project=. scripts/tangle.jl scripts/02_lotka_volterra.jl
```

```
run:
```

```
    julia --project=. scripts/01_sir.jl
```

```
    julia --project=. scripts/02_lotka_volterra.jl
```

notebooks:

```
jupyter nbconvert --to notebook --execute --inplace \  
--ExecutePreprocessor.kernel_name=julia-lab02-1.11 \  
--ExecutePreprocessor.timeout=900 \  
notebooks/01_sir/01_sir.ipynb  
jupyter nbconvert --to notebook --execute --inplace \  
--ExecutePreprocessor.kernel_name=julia-lab02-1.11 \  
--ExecutePreprocessor.timeout=900 \  
notebooks/02_lotka_volterra/02_lotka_volterra.ipynb
```

docs:

```
quarto render markdown/01_sir/01_sir.qmd --to html --embed-resources  
quarto render markdown/02_lotka_volterra/02_lotka_volterra.qmd --to html --embed-resources
```

test:

```
julia --project=. test/runtests.jl
```

all: tangle run notebooks docs test

Листинг 9 — Управляющие цели labs/lab02/project/Makefile

Отчёт собирается отдельной командой make в каталоге labs/lab02/report. Презентация собирается в каталоге labs/lab02/presentation; там формируются самодостаточный HTML и версия PPTX для просмотра в PowerPoint.

9. Выводы

В лабораторной работе реализованы и исследованы две нелинейные модели.

Для SIR подтверждён пороговый характер распространения инфекции. В базовом случае $R_0 = 2$, максимум числа инфицированных равен приблизительно 158,45 и достигается на 17,5-й день. Сканирование числа контактов показало, что при $R_0 \leq 1$ выраженный пик отсутствует, а рост R_0 приводит к более ранней и высокой эпидемической волне. Суммарная численность сохраняется с дрейфом менее 10^{-12} .

Для модели Лотки–Вольтерры найдена стационарная точка (30,5), построены временные и фазовые траектории, оценён период колебаний 37,6889. Разные начальные условия создают семейство замкнутых орбит вокруг общего равновесия. Первый интеграл сохраняется с максимальным отклонением порядка 10^{-11} .

Проект содержит литературные и чистые Julia-скрипты, CSV-данные, графики, QMD-документы, выполненные IPYNB и автоматические тесты. Все десять проверок пройдены, поэтому вычислительные результаты согласуются с теоретическими свойствами обеих моделей.

Источники

1. Kermack W. O., McKendrick A. G. A Contribution to the Mathematical Theory of Epidemics // Proceedings of the Royal Society A. 1927. Vol. 115. P. 700–721. DOI: 10.1098/rspa.1927.0118.
2. Lotka A. J. Elements of Physical Biology. Baltimore: Williams & Wilkins,

3. Volterra V. Fluctuations in the Abundance of a Species considered Mathematically // Nature. 1926. Vol. 118. P. 558–560.
4. SciML. DifferentialEquations.jl Documentation. <https://docs.sciml.ai/DiffEqDocs/stable/>.
5. DrWatson.jl Documentation. <https://juliadynamics.github.io/DrWatson.jl/>.
6. Literate.jl Documentation. <https://fredrikekre.github.io/Literate.jl/>.
7. Quarto Documentation. <https://quarto.org/docs/>.